



**Universidad
Continental**

TEMA

"App educativo sobre ahorro y consumo responsable de agua"

ASIGNATURA

Análisis y Diseño de Software

NRC

62152

DOCENTE

Rosario Delia Osorio Contreras

INTEGRANTES

- Capani Paintan Brayan Ronaldo (100%)
- Meza Miranda Jhosue Alejandro (100%)
- Gonzales Castro Noel Eduardo (0%)
- Huamani Madueño Giovanni Jesus (0%)

Perú-2025

ÍNDICE DE CONTENIDOS

1. **CAPÍTULO I: FUNDAMENTOS DEL PROYECTO**
 - 1.1. Descripción de la Realidad Problemática
 - 1.2. Objetivos del Proyecto y Vinculación ODS
 - 1.3. Alcance y Stakeholders
2. **CAPÍTULO II: ANÁLISIS DE REQUERIMIENTOS**
 - 2.1. Identificación de Necesidades
 - 2.2. Requerimientos Funcionales (RF)
 - 2.3. Requerimientos No Funcionales (RNF)
3. **CAPÍTULO III: MODELADO DEL SISTEMA (ANÁLISIS)**
 - 3.1. Modelo de Casos de Uso (Refinado)
 - 3.2. Especificación de Actores y Escenarios
 - 3.3. Modelo de Procesos (Diagrama de Actividad)
 - 3.4 Diagrama de actividad UML:
 - 3.5. Diseño de Interacción (Prototipado inicial)
4. **CAPÍTULO IV: DISEÑO DE ARQUITECTURA DE SOFTWARE**
 - 4.1. Arquitectura Lógica (Capas y MVC)
 - 4.2. Patrones de Diseño Aplicados (Singleton, DAO)
 - 4.3. Modelo Estructural (Diagrama de Clases)
 - 4.4. Modelo de Comportamiento (Diagrama de Secuencia)
 - 4.5. Arquitectura de Despliegue e Infraestructura
5. **CAPÍTULO V: DISEÑO DE DATOS E IMPLEMENTACIÓN**
 - 5.1. Modelo Entidad-Relación y Normalización
 - 5.2. Lógica Transaccional (Scripts y Triggers)
 - 5.3. Interfaz de Usuario (Prototipos)
 - 5.4. Matriz de Trazabilidad de Requisitos
6. **CONCLUSIONES Y RECOMENDACIONES**
7. **REFERENCIAS BIBLIOGRÁFICAS**
8. **ANEXOS**

CAPÍTULO I: FUNDAMENTOS DEL PROYECTO

1.1. Descripción de la Realidad Problemática

La gestión ineficiente del recurso hídrico representa uno de los desafíos más críticos del siglo XXI. Según informes de la UNESCO (2020), la escasez de agua afecta a más del 40% de la población mundial. En el contexto local, se ha identificado una brecha tecnológica significativa: las campañas de concientización actuales son pasivas y carecen de mecanismos para medir el cambio conductual real en la población. No existen herramientas interactivas que validen y recompensen el ahorro de agua doméstico, lo que resulta en una desconexión entre el conocimiento teórico y la práctica diaria.

1.2. Objetivos y Vinculación ODS

El objetivo técnico es desarrollar una **plataforma móvil basada en gamificación** que instrumentalice el **ODS 6: Agua Limpia y Saneamiento**.

- **Meta Técnica:** Implementar un motor de reglas que transforme acciones físicas (ahorro de agua) en activos digitales (puntos e insignias).
- **Impacto Social:** Apoyar la meta 6.b de la ONU mediante la participación comunitaria en la gestión del agua y el saneamiento.

CAPÍTULO II: ANÁLISIS DE REQUERIMIENTOS

2.1. Requerimientos Funcionales (RF)

El sistema debe cumplir con las siguientes funciones críticas para la operación:

- **RF01 - Gestión de Identidad:** "El sistema permitirá el registro y autenticación de usuarios mediante credenciales cifradas (correo/contraseña) y/o servicios de identidad federada (Google OAuth), garantizando la unicidad de las cuentas."
- **RF03 - Orquestación de Retos:** El sistema generará y desplegará un reto diario único sincronizado con el servidor, permitiendo la validación de su cumplimiento para evitar duplicidad de registros.
- **RF05 - Motor de Gamificación:** El sistema calculará automáticamente el puntaje acumulado y disparará eventos de asignación de insignias al superar los umbrales definidos.
- **RF08 - Administración de Contenidos (CMS):** El módulo administrativo permitirá operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre las lecciones y quizzes.

2.2. Requerimientos No Funcionales (RNF)

- **RNF01 - Usabilidad (HCI):** La interfaz debe cumplir con principios heurísticos de consistencia y visibilidad, asegurando una curva de aprendizaje mínima.
- **RNF02 - Rendimiento:** El tiempo de carga de los módulos educativos no debe exceder los 2 segundos mediante el uso de estrategias de caché local.

- **RNF05 - Disponibilidad:** El sistema debe operar bajo un esquema *Offline-First*, permitiendo funcionalidad básica sin conexión y sincronización posterior.

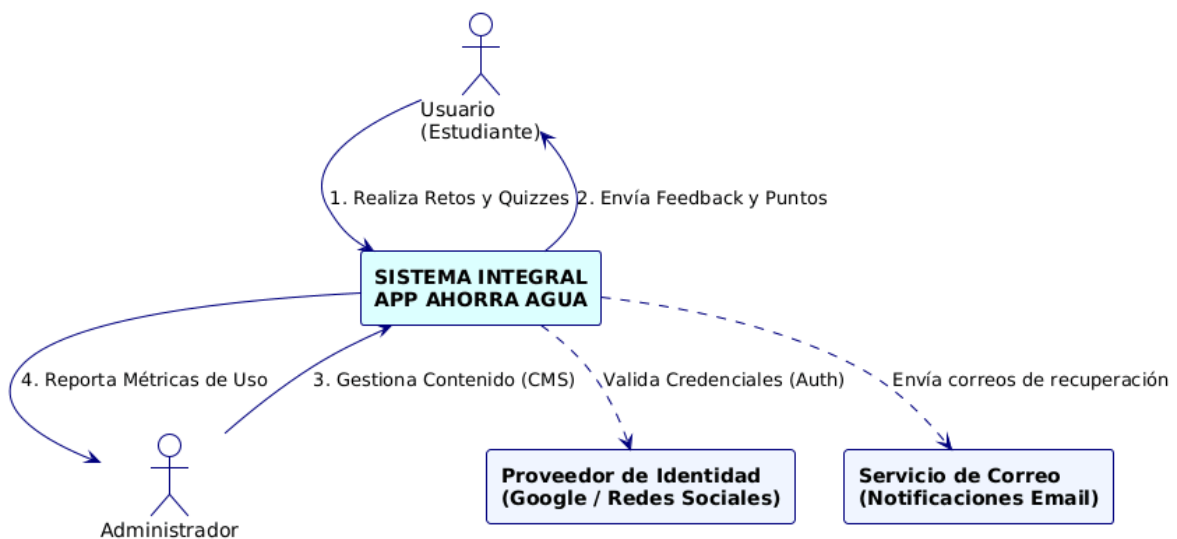
2.4. Requerimientos de Dominio (Reglas de Negocio)

Estas reglas definen las restricciones lógicas y condiciones operativas propias del contexto educativo y de gamificación del proyecto:

- **RD01 - Unicidad del Reto Diario:** El sistema debe asegurar que el "Reto del Día" sea idéntico para todos los usuarios de la misma zona horaria y solo pueda ser marcado como completado una única vez por cuenta en un periodo de 24 horas (UTC-5), impidiendo la explotación de puntos mediante repetición de tareas.
- **RD02 - Estandarización del Algoritmo de Puntuación:** El cálculo de recompensas debe seguir una lógica determinista estricta para garantizar la equidad:
 - **Reto Diario Completado:** +10 puntos base.
 - **Quiz Aprobado (>60%):** +20 puntos de experiencia.
 - **Bono de Excelencia:** +5 puntos adicionales si el quiz se aprueba con nota perfecta (100%) en el primer intento.
- **RD03 - Lógica de Desbloqueo de Insignias:** La asignación de insignias es automática y acumulativa. El sistema disparará el evento de desbloqueo inmediatamente cuando el atributo `puntos_acumulados` del usuario cruce los umbrales definidos en la base de datos (Ejemplo: Insignia "Guardián del Agua" al alcanzar los 100 puntos), sin requerir intervención manual de un administrador.

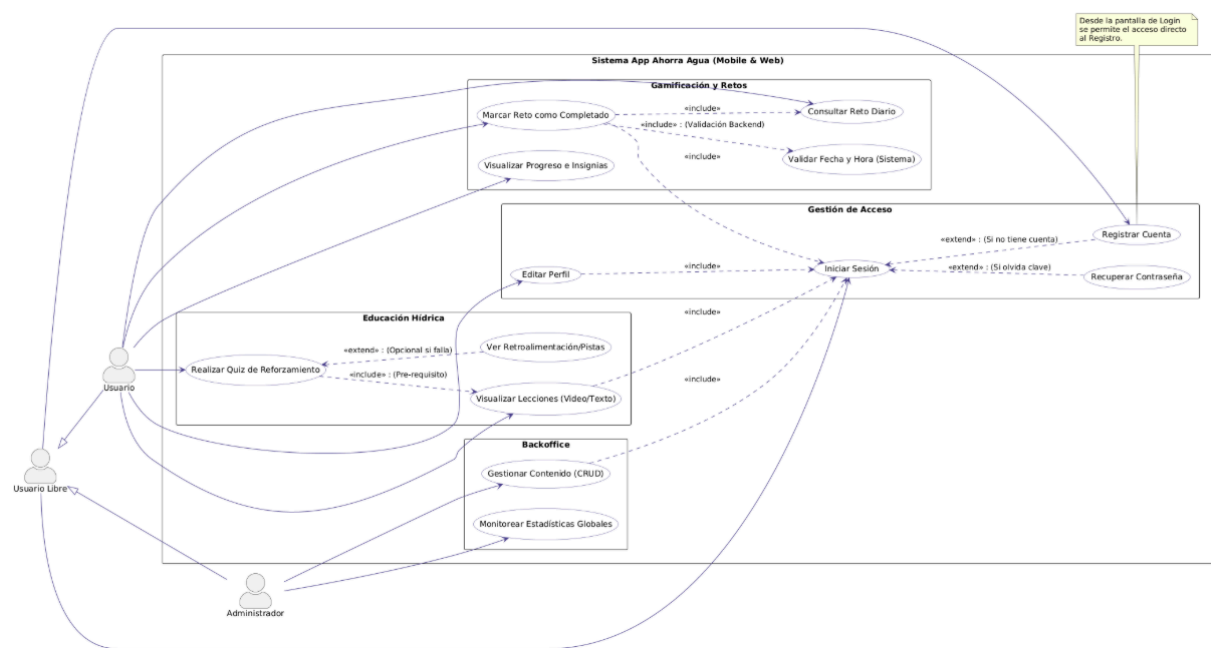
CAPÍTULO III: MODELADO DEL SISTEMA (ANÁLISIS)

Diagrama de Contexto: Sistema App Ahorra Agua



3.1. Modelo de Casos de Uso:

El diagrama de casos de uso ha sido estructurado para reflejar las dependencias funcionales del sistema. Se han aplicado relaciones de inclusión (<<include>>) para garantizar que procesos críticos como la "Gestión de Progreso" hereden obligatoriamente la validación de seguridad. Asimismo, se modelan extensiones (<<extend>>) para funcionalidades de soporte pedagógico en los quizzes.



3.2. Especificación de Actores y Escenarios

A continuación se definen los roles que interactúan con el sistema, estableciendo sus permisos y responsabilidades:

Actor	Descripción y Responsabilidad
Usuario Libre	Persona que ha descargado la aplicación pero aún no posee credenciales. Su interacción se limita a las pantallas de aterrizaje (Landing), Registro y Login.
Usuario (Registrado)	Actor principal del sistema. Tiene acceso al núcleo de gamificación: visualizar lecciones, completar el reto diario, rendir

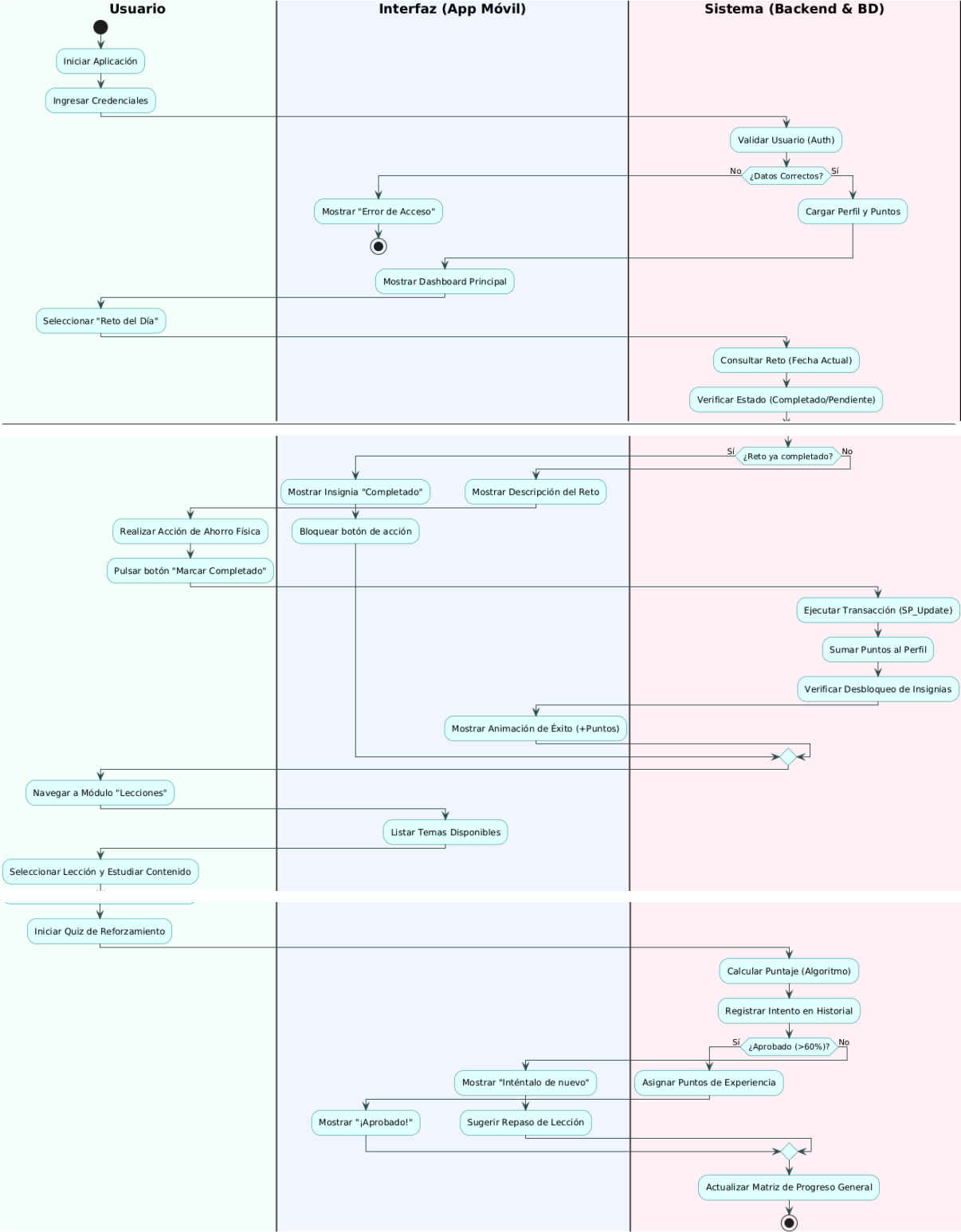
	quizzes y acumular insignias. Sus datos de progreso se persisten en la base de datos.
Administrador	Encargado de la gestión de contenidos (CMS). Tiene privilegios para crear, editar o eliminar lecciones, quizzes y retos del banco global, así como visualizar reportes estadísticos de uso.

3.3. Modelo de Procesos (Diagrama de Actividad)

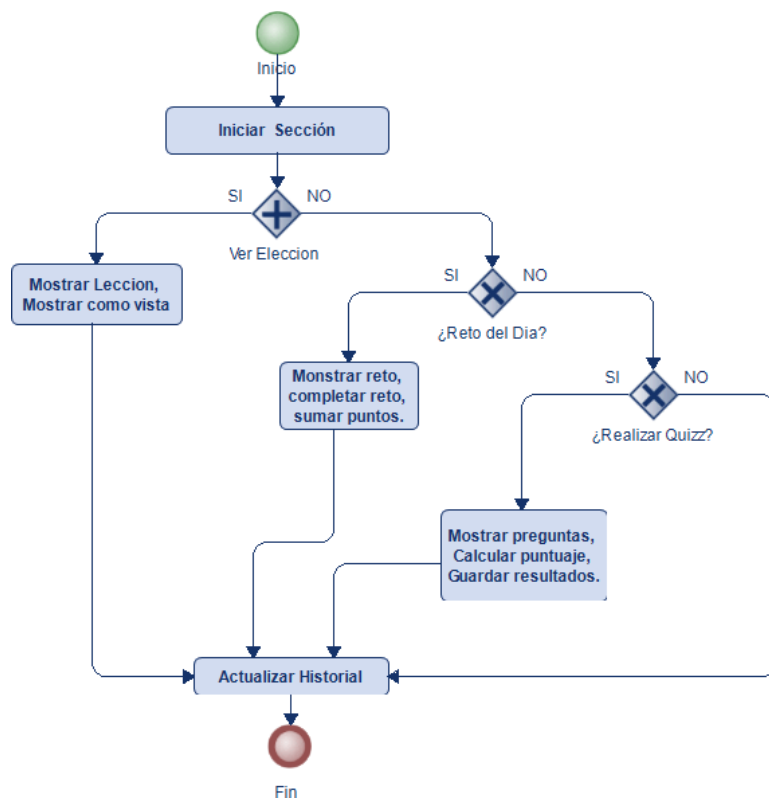
El flujo de proceso principal, denominado "**Ciclo de Aprendizaje Diario**", se ha modelado utilizando un diagrama de actividad con carriles (swimlanes). Esta estructura permite visualizar claramente la interacción entre las tres capas del sistema:

1. **Carril de Usuario:** Representa las acciones manuales y decisiones del estudiante (ej. "Realizar Acción de Ahorro").
2. **Carril de Interfaz (App Móvil):** Detalla la respuesta visual de la aplicación, cumpliendo con el RNF01 de usabilidad (ej. "Mostrar Animación de Éxito").
3. **Carril de Sistema (Backend & BD):** Ilustra la lógica de negocio invisible al usuario, como las validaciones condicionales (verificar si el reto ya fue completado) y las transacciones de base de datos (SP_Update).

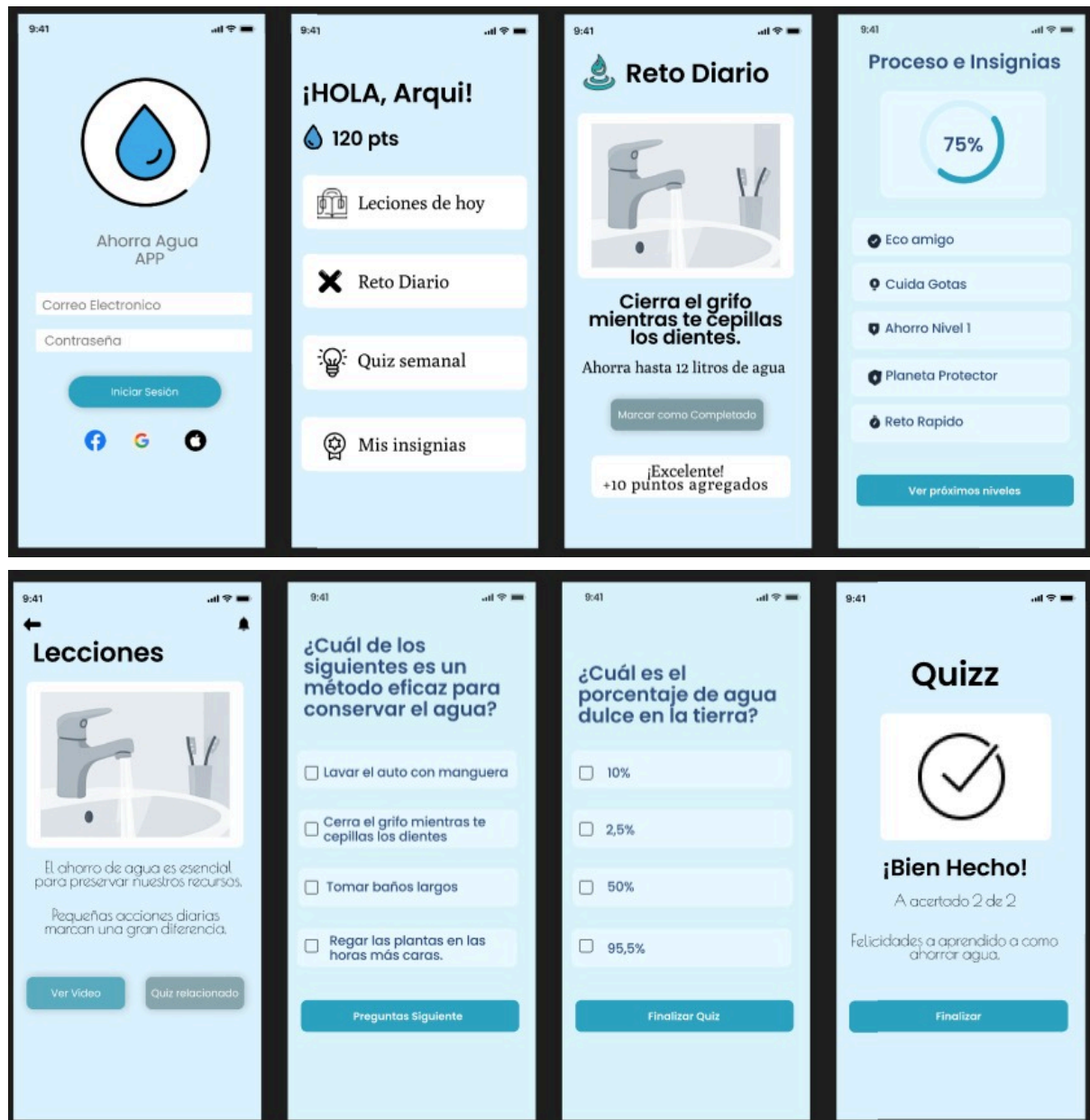
Modelo de Procesos: Ciclo de Aprendizaje y Retos Diarios



3.4 Diagrama de actividad UML:



3.5: Diseño de Interacción (Prototipado inicial):



CAPÍTULO IV: DISEÑO DE ARQUITECTURA DE SOFTWARE

4.1. Arquitectura Lógica

Se ha implementado una arquitectura en capas basada en el patrón **MVC (Modelo-Vista-Controlador)** para garantizar la escalabilidad y mantenibilidad:

- **Vista (Presentación):** Responsable únicamente de la renderización de la interfaz (UI) y captura de eventos. No procesa datos.
- **Controlador (Negocio):** Contiene la lógica del juego educativo, validaciones de fechas y cálculo de algoritmos de puntuación. Actúa como intermediario de seguridad.

- **Modelo (Datos):** Gestiona la persistencia y la integridad referencial de la información mediante objetos de acceso a datos (DAO).

4.2. Patrones de Diseño Aplicados

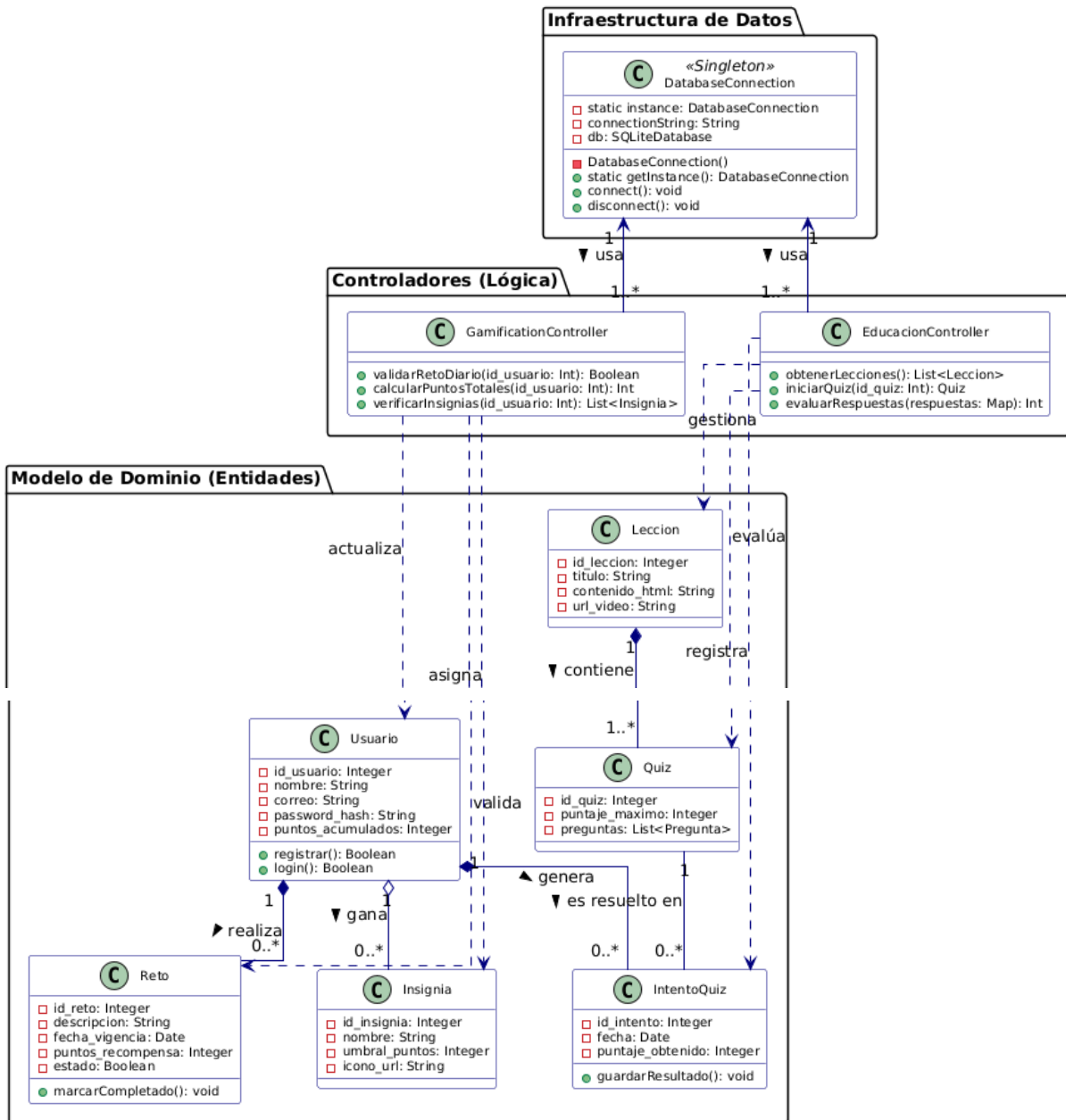
Para solventar problemas recurrentes de ingeniería, se aplicaron los siguientes patrones:

1. **Singleton:** Implementado en la clase `DatabaseConnection`. Su propósito es restringir la instanciación de la conexión a la base de datos a un **único objeto global**. Esto es vital en dispositivos móviles para evitar el desbordamiento de memoria por múltiples conexiones abiertas simultáneamente.
2. **Observer:** Utilizado en el sistema de Notificaciones. Permite que el módulo de "Insignias" observe pasivamente el acumulador de "Puntos", reaccionando automáticamente cuando se alcanza una meta, desacoplando así la lógica de ambos componentes.

4.3. Diagrama de Clases (Diseño Estructural)

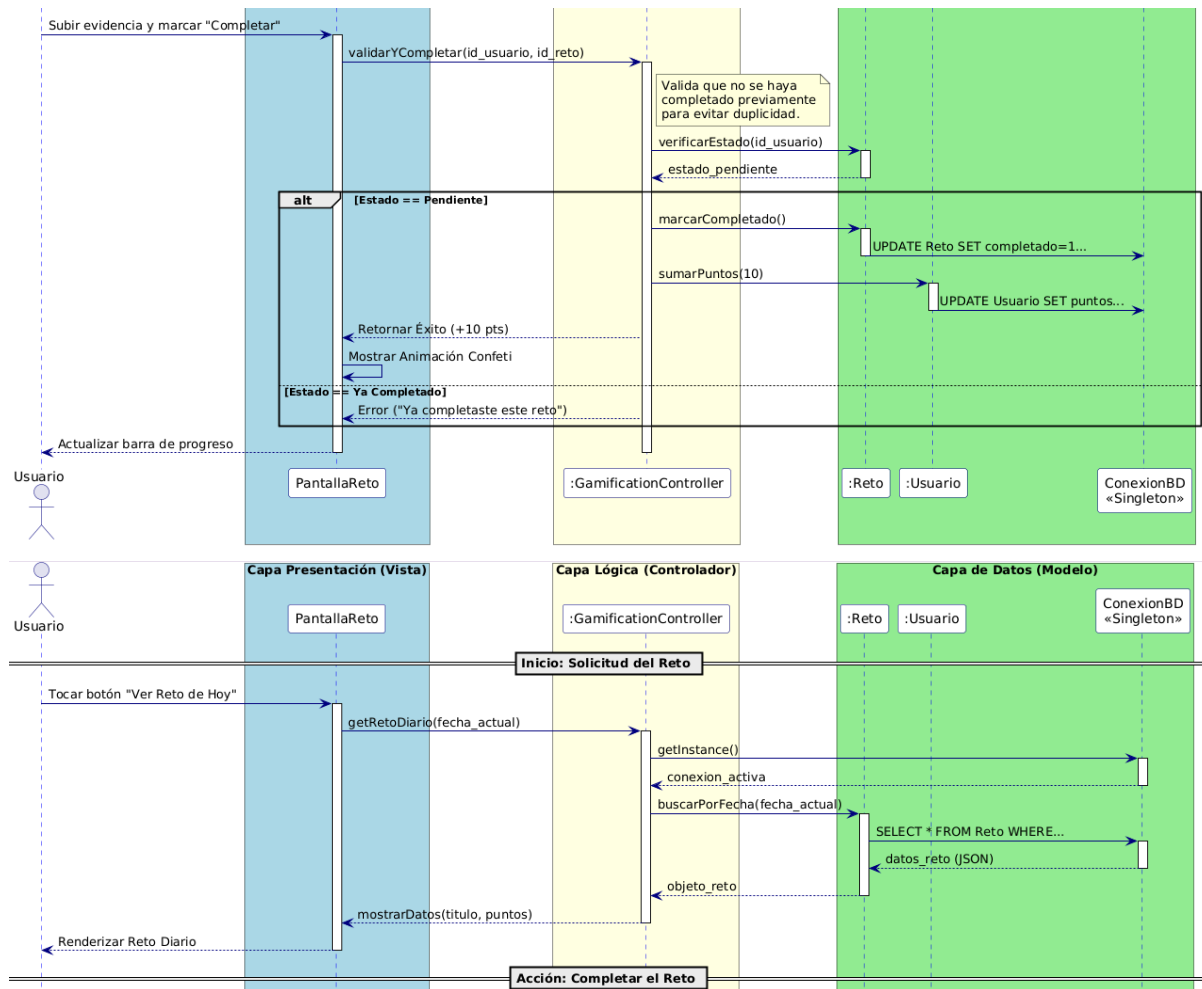
El siguiente diagrama detalla la estructura estática del sistema, evidenciando la aplicación del patrón Singleton y la tipificación fuerte de las entidades.

Diagrama de Clases: Arquitectura Lógica y Persistencia



4.4. Diagrama de Secuencia (Diseño de Comportamiento)

Este modelo detalla el flujo de mensajes para el caso de uso crítico **"Completar Reto"**, demostrando la orquestación técnica entre la Vista, el Controlador y la Base de Datos para garantizar la consistencia transaccional.

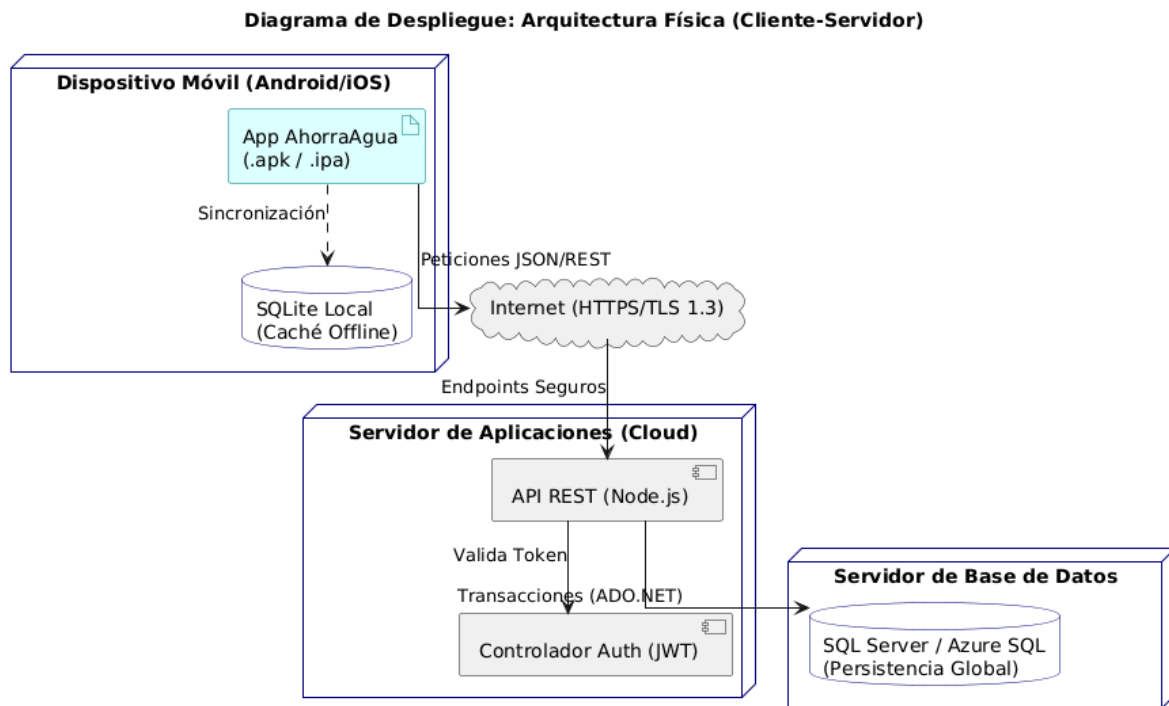


4.5. Arquitectura de Despliegue e Infraestructura:

El sistema se despliega bajo un modelo **Cliente-Servidor Híbrido** diseñado para operar en entornos con conectividad intermitente (Offline-First). La infraestructura física se compone de los siguientes nodos:

1. **Nodo Cliente (Dispositivo Móvil):** El artefacto de software (**.apk**) se instala en dispositivos Android/iOS. Este nodo incluye una base de datos local **SQLite** que actúa como caché de persistencia, permitiendo al estudiante acceder a lecciones y completar retos sin conexión a internet activa.
2. **Canal de Comunicación (Red):** El intercambio de datos se realiza sobre el protocolo **HTTPS** utilizando el estándar **API REST**. Los mensajes se serializan en formato **JSON**, garantizando una transmisión ligera y compatible con cualquier plataforma.
3. **Nodo Servidor (Nube):**
 - **Servidor de Aplicaciones:** Aloja la lógica de negocio y los controladores de autenticación (JWT). Actúa como pasarela segura para las peticiones del cliente.

- **Servidor de Base de Datos:** Resguarda la información centralizada en un motor relacional (SQL Server / Azure SQL), garantizando la integridad definitiva de los puntajes y credenciales.



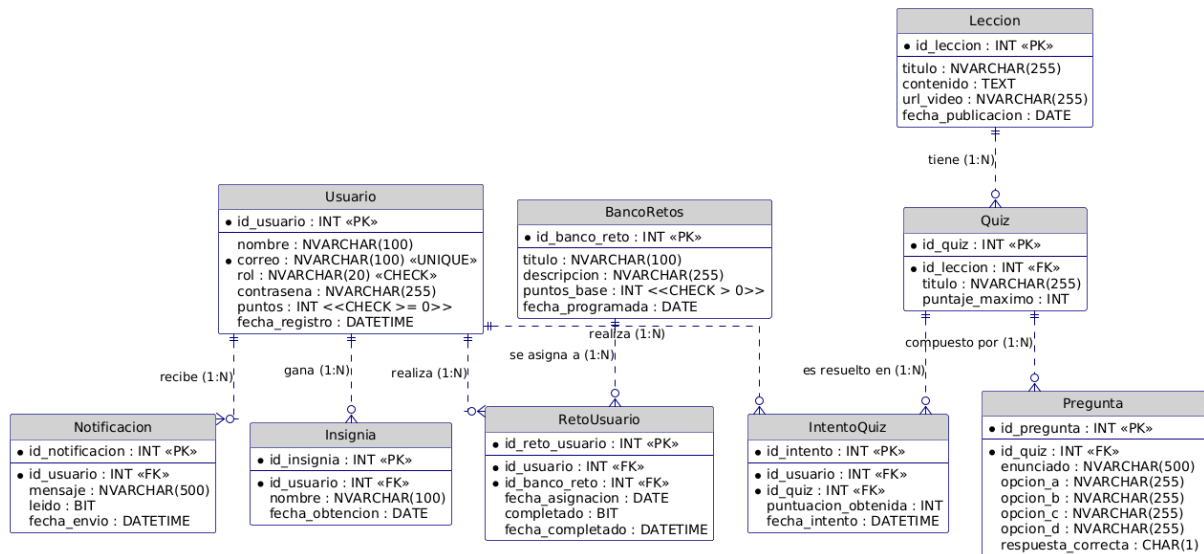
CAPÍTULO V: DISEÑO DE DATOS E IMPLEMENTACIÓN

5.1. Modelo de Base de Datos y Normalización

El esquema relacional ha sido diseñado cumpliendo la **Tercera Forma Normal (3FN)** para eliminar redundancias y anomalías de actualización.

- **Normalización de Retos:** Se observa claramente la separación entre **BancoRetos** (Catálogo maestro) y **RetoUsuario** (Transaccional), eliminando la redundancia de datos.
- **Detalle de Evaluación:** La entidad **Pregunta** permite almacenar múltiples ítems por Quiz, y la entidad **IntentoQuiz** permite que un usuario pueda reintentar la evaluación guardando un historial completo, no solo la última nota.
- **Integridad:** Las llaves foráneas (FK) aseguran que no existan registros huérfanos (por ejemplo, una pregunta sin quiz o un progreso sin usuario).

Diagrama E-R Final: Base de Datos Normalizada (3FN)



5.2. Lógica en Base de Datos (Scripts)

Se ha trasladado parte de la lógica de negocio a la base de datos para optimizar el rendimiento:

- **Procedimientos Almacenados:** Se utiliza `sp_actualizar_puntos` para encapsular la lógica de actualización, previniendo inyecciones SQL.
- **Triggers:** Se implementó `trg_asignar_puntos_reto` para automatizar la suma de puntos tras completar un reto, garantizando consistencia atómica de los datos.

5.3. Interfaz de Usuario (Prototipos):

"A continuación se presenta el diseño final de la interfaz gráfica de usuario (GUI) en alta fidelidad. Estas pantallas materializan los requerimientos funcionales y no funcionales definidos previamente, implementando una arquitectura de información que prioriza la usabilidad (RNF01) y la accesibilidad. El diseño visual utiliza una paleta de colores coherente con la temática hídrica y patrones de navegación estándar (Material Design) para reducir la carga cognitiva del estudiante, facilitando el acceso intuitivo a los módulos de retos, evaluación y seguimiento de progreso."

5.4. Matriz de Trazabilidad de Requisitos

La siguiente matriz valida que cada requerimiento inicial ha sido cubierto por un componente de software y un artefacto de código específico, asegurando la calidad del entregable final.

ID Req	Descripción del Requerimiento	Caso de Uso Asociado	Componente del Sistema (Clase)	Artefacto de Código / BD
RF01	Gestión de Identidad	UC-01 Registrar Cuenta	Usuario, AuthController	Tabla Usuario
RF03	Orquestación de Reto Diario	UC-06 Completar Reto	Reto, GamificationController	SP sp_actualizar_puntos
RF04	Realización de Quiz	UC-05 Realizar Quiz	Quiz, IntentoQuiz	Tabla Quiz, SP sp_registrar_intento
RF05	Motor de Gamificación	UC-07 Consultar Progreso	Insignia, GamificationController	Trigger trg_asignar_puntos
RF08	Administración de Contenidos	UC-08 Gestionar Contenido	AdminController	Script CRUD

RNF02	Rendimiento (< 2s)	N/A	DatabaseConnection	Patrón Singleton
--------------	--------------------	-----	--------------------	------------------

CONCLUSIONES

1. **Robustez Técnica:** La implementación de una arquitectura en capas desacopladas y el uso del patrón Singleton aseguran que la aplicación sea escalable y eficiente en el uso de recursos móviles, cumpliendo con los estándares de ingeniería de software modernos.
2. **Impacto Medible en ODS 6:** A diferencia de soluciones puramente informativas, el sistema propuesto permite cuantificar el compromiso del usuario mediante métricas de trazabilidad (retos completados), alineando la tecnología directamente con los indicadores de sostenibilidad de la ONU.
3. **Consistencia de Datos:** La aplicación de reglas de normalización (3FN) y lógica a nivel de base de datos (Triggers) garantiza la integridad de la información del usuario, elemento crítico para la credibilidad del sistema de gamificación.

REFERENCIAS BIBLIOGRÁFICAS

1. Organización de las Naciones Unidas. (2015). *Objetivos de Desarrollo Sostenible: ODS 6 Agua limpia y saneamiento*. ONU.
2. UNESCO. (2020). *Informe mundial de las Naciones Unidas sobre el desarrollo de los recursos hídricos 2020*. UNESCO.
3. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. (Referencia teórica para Patrones Singleton/Observer).

ANEXOS

Enlace al Repositorio de Código (GitHub):

Script SQL Completo:

-- =====

-- **SISTEMA:** App Educativa "Ahorra Agua" (ODS 6)

-- **FECHA:** Noviembre 2025

-- **ARQUITECTURA:** Base de Datos Relacional Normalizada (3FN)

-- **AUTORES:** Capani, Meza, Gonzales, Huamani

-- =====

-- 1. CREACIÓN DE TABLAS (DDL)

-- Tabla: Usuario

-- Mejora: Se añade restricción CHECK para evitar puntos negativos

```
CREATE TABLE Usuario (  
    id_usuario INT IDENTITY(1,1) PRIMARY KEY,  
    nombre NVARCHAR(100) NOT NULL,  
    correo NVARCHAR(100) UNIQUE NOT NULL,  
    rol NVARCHAR(20) CHECK(rol IN ('Estudiante', 'Administrador')) NOT NULL,  
    contraseña NVARCHAR(255) NOT NULL, -- Hash SHA-256  
    puntos INT DEFAULT 0 CHECK (puntos >= 0),  
    fecha_registro DATETIME DEFAULT GETDATE()  
);
```

-- Tabla: Lecciones Educativas

```
CREATE TABLE Leccion (  
    id_leccion INT IDENTITY(1,1) PRIMARY KEY,
```

```
    titulo NVARCHAR(255) NOT NULL,  
    contenido TEXT NOT NULL, -- Contenido HTML/Texto  
    url_video NVARCHAR(255), -- Enlace a recurso multimedia  
    fecha_publicacion DATE DEFAULT GETDATE()  
);
```

-- Tabla Maestra: Banco de Retos (Catálogo)

-- Aquí se definen los retos disponibles. No se repiten datos.

```
CREATE TABLE BancoRetos (  
    id_banco_reto INT IDENTITY(1,1) PRIMARY KEY,  
    titulo NVARCHAR(100) NOT NULL,  
    descripcion NVARCHAR(255) NOT NULL,  
    puntos_base INT NOT NULL CHECK (puntos_base > 0),  
    fecha_programada DATE NULL -- Opcional, si es para un día específico  
);
```

-- Tabla Transaccional: Progreso de Retos por Usuario

-- Vincula usuarios con retos y controla el estado de cumplimiento

```
CREATE TABLE RetoUsuario (  
    id_reto_usuario INT IDENTITY(1,1) PRIMARY KEY,  
    id_usuario INT NOT NULL,  
    id_banco_reto INT NOT NULL,  
    fecha_asignacion DATE DEFAULT GETDATE(),
```

completado BIT DEFAULT 0,
fecha_completado DATETIME NULL,

CONSTRAINT FK_RetoU_Usuario FOREIGN KEY (id_usuario)
REFERENCES Usuario(id_usuario) ON DELETE CASCADE,
CONSTRAINT FK_RetoU_Banco FOREIGN KEY (id_banco_reto)
REFERENCES BancoRetos(id_banco_reto)

);

-- Tabla: Quiz (Cabecera de Evaluación)

CREATE TABLE Quiz (
id_quiz INT IDENTITY(1,1) PRIMARY KEY,
id_leccion INT NOT NULL,
titulo NVARCHAR(255) NOT NULL,
puntaje_maximo INT DEFAULT 20,
CONSTRAINT FK_Quiz_Leccion FOREIGN KEY (id_leccion)
REFERENCES Leccion(id_leccion) ON DELETE CASCADE
);

-- Tabla: Preguntas (Detalle del Quiz)

-- Almacena las preguntas individuales vinculadas al Quiz

CREATE TABLE Pregunta (
id_pregunta INT IDENTITY(1,1) PRIMARY KEY,
id_quiz INT NOT NULL,
enunciado NVARCHAR(500) NOT NULL,
opcion_a NVARCHAR(255) NOT NULL,

```
opcion_b NVARCHAR(255) NOT NULL,  
opcion_c NVARCHAR(255) NOT NULL,  
opcion_d NVARCHAR(255) NOT NULL,  
respuesta_correcta CHAR(1) NOT NULL CHECK (respuesta_correcta IN ('A', 'B', 'C', 'D')),
```

```
CONSTRAINT FK_Pregunta_Quiz FOREIGN KEY (id_quiz)  
REFERENCES Quiz(id_quiz) ON DELETE CASCADE
```

```
);
```

-- Tabla: Intentos de Quiz (Historial)

```
CREATE TABLE IntentoQuiz (  
    id_intento INT IDENTITY(1,1) PRIMARY KEY,  
    id_usuario INT NOT NULL,  
    id_quiz INT NOT NULL,  
    puntuacion_obtenida INT CHECK (puntuacion_obtenida >= 0),  
    fecha_intento DATETIME DEFAULT GETDATE(),  
    CONSTRAINT FK_Intento_Usuario FOREIGN KEY (id_usuario)  
        REFERENCES Usuario(id_usuario) ON DELETE CASCADE,  
    CONSTRAINT FK_Intento_Quiz FOREIGN KEY (id_quiz)  
        REFERENCES Quiz(id_quiz)  
);
```

-- Tabla: Insignias (Gamificación)

```
CREATE TABLE Insignia (  
    id_insignia INT IDENTITY(1,1) PRIMARY KEY,  
    id_usuario INT NOT NULL,  
    nombre NVARCHAR(100) NOT NULL,  
    fecha_obtencion DATE DEFAULT GETDATE(),  
    CONSTRAINT FK_Insignia_Usuario FOREIGN KEY (id_usuario)  
        REFERENCES Usuario(id_usuario) ON DELETE CASCADE  
);
```

-- Tabla: Notificaciones

```
CREATE TABLE Notificacion (  
    id_notificacion INT IDENTITY(1,1) PRIMARY KEY,  
    id_usuario INT NOT NULL,  
    mensaje NVARCHAR(500) NOT NULL,  
    leido BIT DEFAULT 0,  
    fecha_envio DATETIME DEFAULT GETDATE(),  
    CONSTRAINT FK_Notif_Usuario FOREIGN KEY (id_usuario)  
        REFERENCES Usuario(id_usuario) ON DELETE CASCADE  
);
```

-- 2. OPTIMIZACIÓN (ÍNDICES PARA RNF02 - RENDIMIENTO)

-- Justificación Técnica: Acelera las búsquedas más frecuentes

```
CREATE INDEX idx_usuario_correo ON Usuario(correo);  
CREATE INDEX idx_reto_progreso ON RetoUsuario(id_usuario, fecha_asignacion);  
CREATE INDEX idx_quiz_leccion ON Quiz(id_leccion);
```

-- 3. PROCEDIMIENTOS ALMACENADOS (LÓGICA DE NEGOCIO EN BD)

GO

-- SP: Actualizar puntos de manera segura

CREATE PROCEDURE sp_actualizar_puntos (

 @usuario_id INT,

 @puntos_ganados INT

)

AS

BEGIN

 SET NOCOUNT ON;

 UPDATE Usuario

 SET puntos = puntos + @puntos_ganados

 WHERE id_usuario = @usuario_id;

END;

GO

-- SP: Registrar un intento de quiz

CREATE PROCEDURE sp_registrar_intento (

 @usuario_id INT,

 @quiz_id INT,

 @puntaje INT

)

AS

BEGIN

```
SET NOCOUNT ON;
```

```
INSERT INTO IntentoQuiz (id_usuario, id_quiz, puntuacion_obtenida)
```

```
VALUES (@usuario_id, @quiz_id, @puntaje);
```

```
-- Lógica extra: Bonificación por excelencia
```

```
IF @puntaje = 20
```

```
BEGIN
```

```
    EXEC sp_actualizar_puntos @usuario_id, 5; -- Bonus de 5 puntos
```

```
END
```

```
END;
```

```
GO
```

-- 4. TRIGGERS (AUTOMATIZACIÓN)

```
-- Trigger: Asignar puntos automáticamente al completar reto
```

```
CREATE TRIGGER trg_completar_reto
```

```
ON RetoUsuario
```

```
AFTER UPDATE
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    -- Solo si cambió de 0 a 1 (se completó)
```

```
    IF UPDATE(completado)
```

```
BEGIN
```

```
    DECLARE @puntos INT, @uid INT;
```

```

-- Obtenemos los puntos base desde la tabla maestra BancoRetos

SELECT @puntos = b.puntos_base, @uid = i.id_usuario

FROM inserted i

INNER JOIN BancoRetos b ON i.id_banco_reto = b.id_banco_reto

WHERE i.completado = 1;

IF @puntos IS NOT NULL

BEGIN

    -- 1. Actualizar Puntos

    UPDATE Usuario SET puntos = puntos + @puntos WHERE id_usuario = @uid;

    -- 2. Notificar al Usuario (Feedback Inmediato)

    INSERT INTO Notificacion (id_usuario, mensaje)

        VALUES (@uid, '¡Felicitades! Ganaste ' + CAST(@puntos AS NVARCHAR) + '

puntos por completar el reto.');
```

END

END

END;

GO

Evidencias de Gestión Ágil (Jira)

